

Write a query to calculate the rolling average sales for each day over the past 7 days.

LinkedIn@AshishZope

```
SELECT sale_date, sales_amount,  
       AVG(sales_amount) OVER (ORDER BY sale_date ROWS BETWEEN 6 PRECEDING AND  
                               CURRENT ROW) AS rolling_avg_7_days  
FROM daily_sales;
```

Explanation:

- The AVG() function calculates the rolling average over the past 7 days.
- The ROWS BETWEEN 6 PRECEDING AND CURRENT ROW ensures that only the last 7 days' sales are considered.

Write a query to find the second-highest salary in each department.

LinkedIn@AshishZope

```
SELECT department_id, MAX(salary) AS second_highest_salary  
FROM employees  
WHERE salary < (  
    SELECT MAX(salary)  
    FROM employees e  
    WHERE e.department_id = employees.department_id  
)  
GROUP BY department_id;
```

Explanation:

- First, the highest salary per department is retrieved. The query then finds the second-highest salary by excluding the maximum salary.

Retrieve all orders placed between 9:00 AM and 5:00 PM.

```
SELECT order_id, order_time
FROM orders
WHERE CAST(order_time AS TIME) BETWEEN '09:00:00' AND '17:00:00';
```

Explanation:

The CAST(order_time AS TIME) ensures comparison is done at the time level. The BETWEEN condition filters orders placed within working hours.

Identify dates where no sales were recorded for each product.

```
SELECT p.product_id, d.date
FROM products p
CROSS JOIN dates d
LEFT JOIN sales s ON p.product_id = s.product_id AND d.date = s.sale_date
WHERE s.sale_date IS NULL;
```

Explanation:

A CROSS JOIN generates all possible product-date combinations. A LEFT JOIN checks for missing sales dates. The WHERE s.sale_date IS NULL filters out dates without sales.

Find customers who made their first order in the last month.

```
SELECT customer_id, MIN(order_date) AS first_order_date
FROM orders
GROUP BY customer_id
HAVING MIN(order_date) >= DATEADD(month, -1, GETDATE());
```

Explanation:

The MIN(order_date) finds the first purchase date. The HAVING clause filters customers who ordered in the last month.

Retrieve customers who have purchased multiple products in the same order.

LinkedIn@AshishZope

```
SELECT order_id, customer_id
FROM order_details
GROUP BY order_id, customer_id
HAVING COUNT(DISTINCT product_id) > 1;
```

Explanation:

The GROUP BY order_id, customer_id groups orders. The HAVING COUNT(DISTINCT product_id) > 1 filters multi-product orders.

Identify orders where the amount is more than twice the average order value.

LinkedIn@AshishZope

```
SELECT order_id, customer_id, order_amount
FROM orders
WHERE order_amount > 2 * (
    SELECT AVG(order_amount) FROM orders);
```

Explanation:

The AVG(order_amount) retrieves the average order value. Orders exceeding twice this value are flagged.

Compute the average number of days between orders for each customer.

```
SELECT customer_id,  
       AVG(DATEDIFF(day, LAG(order_date) OVER (PARTITION BY customer_id ORDER  
BY order_date), order_date)) AS avg_days_between_orders  
FROM orders;
```

Explanation:

The LAG() function retrieves the previous order date. The DATEDIFF() calculates the difference in days. The AVG() computes the overall average.

Identify the highest revenue month per year.

```
SELECT year, month, revenue  
FROM (  
    ...sql  
  
    SELECT year, month, revenue,  
           RANK() OVER (PARTITION BY year ORDER BY revenue DESC) AS rank  
    FROM monthly_revenue  
  ) AS yearly_revenue  
WHERE rank = 1;
```

Explanation:

The RANK() function orders months by revenue within each year. The outer query filters the top month per year.

Find the most popular product based on sales in each category.

```
SELECT category_id, product_name, MAX(sales) AS max_sales  
FROM products  
GROUP BY category_id, product_name  
ORDER BY max_sales DESC;
```

Explanation:

The MAX(sales) retrieves the highest-selling product in each category.

Calculate the average order value (AOV) by month.

[LinkedIn@AshishZope](#)

```
SELECT month, AVG(order_amount) AS avg_order_value
FROM orders
GROUP BY month;
```

Explanation:

The AVG(order_amount) computes the monthly AOV.

Identify products with sales growth from Q1 to Q2.

[LinkedIn@AshishZope](#)

```
SELECT p.product_id,
       q1.sales AS q1_sales, q2.sales AS q2_sales,
       (q2.sales - q1.sales) / NULLIF(q1.sales, 0) * 100 AS growth_rate
FROM (``sql

SELECT product_id, SUM(sales) AS sales FROM sales WHERE quarter = 'Q1' GROUP BY
product_id) q1
JOIN (``sql

SELECT product_id, SUM(sales) AS sales FROM sales WHERE quarter = 'Q2' GROUP BY
product_id) q2
ON q1.product_id = q2.product_id;
```

Explanation:

Two subqueries calculate Q1 and Q2 sales for each product. The main query calculates the percentage growth.

Write a query to identify employees assigned to more than one department.

LinkedIn@AshishZope

```
SELECT employee_id
FROM employee_departments
GROUP BY employee_id
HAVING COUNT(DISTINCT department_id) > 1;
```

Explanation:

The HAVING COUNT(DISTINCT department_id) > 1 filters employees assigned to multiple departments. Q. Find Products with Zero Sales in the Last Quarter

LinkedIn@AshishZope

```
SELECT product_id, product_name
FROM products
WHERE product_id NOT IN (
    ``sql

SELECT DISTINCT product_id
FROM sales
WHERE sale_date >= DATEADD(quarter, -1, GETDATE())
);
```

Explanation:

The query excludes products that had sales in the last quarter.

Rank sales representatives based on quarterly sales.

LinkedIn@AshishZope

```
SELECT sales_rep_id, quarter, total_sales,
       RANK() OVER (PARTITION BY quarter ORDER BY total_sales DESC) AS rank
FROM quarterly_sales;
```

Get Sanjay Kumar PhD's stories in your inbox Join Medium for free to get updates from this writer.

Enter your email [Subscribe](#)

Remember me for faster sign in

Explanation:

The RANK() function assigns rankings based on sales performance.

Write a query to find the month with the highest revenue for each year.

LinkedIn@AshishZope

```
SELECT year, month, revenue
FROM (
    ```sql

 SELECT year, month, revenue,
 RANK() OVER (PARTITION BY year ORDER BY revenue DESC) AS rank
 FROM monthly_revenue
) AS yearly_revenue
WHERE rank = 1;
```

### Explanation:

The RANK() function orders months by revenue. The WHERE rank = 1 ensures only the top month is SELECT ed.

## Identify customers who have never ordered product "P123".

LinkedIn@AshishZope

```
SELECT customer_id
FROM customers
WHERE customer_id NOT IN (
    ```sql

    SELECT DISTINCT customer_id
    FROM orders
    WHERE product_id = 'P123'
);
```

Explanation:

The subquery retrieves all customers who purchased "P123". The outer query finds customers not in this list.

Identify products with sales growth from Q1 to Q2.

LinkedIn@AshishZope

```

SELECT p.product_id,
       q1.sales AS q1_sales, q2.sales AS q2_sales,
       (q2.sales - q1.sales) / NULLIF(q1.sales, 0) * 100 AS growth_rate
FROM (``sql

SELECT product_id, SUM(sales) AS sales FROM sales WHERE quarter = 'Q1' GROUP BY
product_id) q1
JOIN (``sql

SELECT product_id, SUM(sales) AS sales FROM sales WHERE quarter = 'Q2' GROUP BY
product_id) q2
ON q1.product_id = q2.product_id;

```

Explanation:

Two subqueries calculate Q1 and Q2 sales for each product. The main query calculates the percentage growth.

Find customers who have placed at least one order in every month for the last 6 months.

WITH monthly_orders AS (``sql

```

SELECT customer_id, MONTH(order_date) AS order_month, YEAR(order_date) AS order_year
FROM orders WHERE order_date >= DATEADD(month, -6, GETDATE()) GROUP BY customer_id,
MONTH(order_date), YEAR(order_date) )

```



```
SELECT customer_id
FROM monthly_orders
GROUP BY customer_id
HAVING COUNT(DISTINCT order_month) = 6;
```

Explanation:

The CTE (monthly_orders) retrieves distinct order months for each customer. The HAVING COUNT(DISTINCT order_month) = 6 ensures orders exist in all 6 months.

Find employees with the longest continuous employment.

```
SELECT employee_id, hire_date, DATEDIFF(year, hire_date, GETDATE()) AS
tenure_years
FROM employees
ORDER BY tenure_years DESC
LIMIT 1;
```

Explanation:

The DATEDIFF() calculates years of employment. ORDER BY tenure_years DESC LIMIT 1 retrieves the longest-serving employee.

Identify the top 3 customers who have spent the most.

```
SELECT customer_id, SUM(order_amount) AS total_spent
FROM orders
GROUP BY customer_id
ORDER BY total_spent DESC
LIMIT 3;
```

Explanation:

The SUM(order_amount) calculates total spending per customer. The LIMIT 3 retrieves the top spenders.

Calculate the percentage revenue growth between two years.

WITH revenue AS (``sql

SELECT YEAR(order_date) AS order_year, SUM(order_amount) AS total_revenue FROM orders
GROUP BY YEAR(order_date))

LinkedIn@AshishZope

```
SELECT r1.order_year, r1.total_revenue,  
       (r1.total_revenue - r2.total_revenue) / NULLIF(r2.total_revenue, 0) *  
100 AS growth_percentage  
FROM revenue r1  
LEFT JOIN revenue r2 ON r1.order_year = r2.order_year + 1;
```

Explanation:

The CTE (revenue) aggregates yearly sales. The LEFT JOIN pairs each year with its previous year for growth calculation.

Find products that have never been purchased.

LinkedIn@AshishZope

```
SELECT product_id, product_name  
FROM products  
WHERE product_id NOT IN (  
    ``sql  
  
SELECT DISTINCT product_id FROM orders  
);
```

Explanation:

The subquery retrieves all purchased product IDs. The outer query finds products missing from this list.

Find products that have never been out of stock.

LinkedIn@AshishZope

```
SELECT product_id, product_name
FROM products
WHERE product_id NOT IN (
    ``sql

SELECT DISTINCT product_id FROM inventory WHERE stock_quantity = 0
);
```

Explanation:

The subquery retrieves products that have been out of stock. The outer query finds products missing from this list.

Rank employees by salary within their department.

LinkedIn@AshishZope

```
SELECT employee_id, department_id, salary,
       RANK() OVER (PARTITION BY department_id ORDER BY salary DESC) AS
salary_rank
FROM employees;
```

Explanation:

The RANK() function assigns rankings per department. PARTITION BY department_id ensures rankings reset for each department.

Find customers who placed their first-ever order this year.

```
SELECT customer_id, MIN(order_date) AS first_order_date
FROM orders
GROUP BY customer_id
HAVING MIN(order_date) >= DATEFROMPARTS(YEAR(GETDATE()), 1, 1);
```

Explanation:

The MIN(order_date) retrieves each customer's first order. The HAVING clause filters customers with first orders in the current year.

Find customers who haven't placed an order in the last 6 months.

```
SELECT customer_id
FROM customers
WHERE customer_id NOT IN (
    ``sql

SELECT DISTINCT customer_id
FROM orders
WHERE order_date >= DATEADD(month, -6, GETDATE())
);
```

Explanation:

The subquery retrieves customers who have ordered in the last 6 months. The outer query finds customers missing from this list.

Identify the most common time of day when orders are placed.

```
SELECT DATEPART(HOUR, order_time) AS order_hour, COUNT(*) AS order_count
FROM orders
GROUP BY DATEPART(HOUR, order_time)
ORDER BY order_count DESC
LIMIT 1;
```

Explanation:

DATEPART(HOUR, order_time) extracts the order hour. The COUNT(*) groups and sorts by frequency.

Identify the longest continuous order streak per customer.

WITH ordered_dates AS (``sql

SELECT customer_id, order_date, ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date) AS rn FROM orders)

```
SELECT customer_id, COUNT(*) AS consecutive_days
FROM ordered_dates
GROUP BY customer_id, DATEADD(day, -rn, order_date)
ORDER BY consecutive_days DESC
LIMIT 1;
```

Explanation:

The ROW_NUMBER() helps track consecutive order days. The DATEADD(day, -rn, order_date) groups sequential dates Sql

Data Science

Machine Learning

Data Analysis

Artificial Intelligence

3

Sanjay Kumar PhD Written by Sanjay Kumar PhD 2.3K followers · 463 following AI & Data Leader | Data Science | Enterprise Architecture | Data Governance & Cloud Transformation | AI Engineering & Product | Author | Keynote Speaker

Follow Responses (3) Unknown user Write a response

What are your thoughts?

Cancel Respond MockRounds.com MockRounds.com

Aug 20, 2025

Your scenario-based SQL interview questions and answers hit the real-world edges—like anti-join patterns. One micro-tip: prefer NOT EXISTS over NOT IN when NULLs may appear. I turned a few into a quick quiz at...more 1

Reply

Gayathri Gayathri

Jun 1

This is really helpful to me to practise scenario-based questions. Thanks Reply

Sengupta Sengupta

Apr 7

please send some important documents in terms of performance tuning. Reply

More from Sanjay Kumar PhD Top 50 Data Architecture Patterns Interview Questions and Answers Sanjay Kumar PhD Sanjay Kumar PhD

.

Aug 17

Top 50 Data Architecture Patterns Interview Questions and Answers Data Engineering, Lakehouse, Medallion, Streaming, Data Mesh, Data Vault & Modern Platform Architecture

57 1 Advanced SQL Interview Questions and Answers Sanjay Kumar PhD Sanjay Kumar PhD

Mar 26, 2025

Advanced SQL Interview Questions and Answers

1. What is the difference between RANK(), DENSE_RANK(), and ROW_NUMBER()? 152 5 2
Top 30 Harness Engineering, Loop Engineering & Graph Engineering Interview Questions and Answers Sanjay Kumar PhD Sanjay Kumar PhD

Aug 7

Top 30 Harness Engineering, Loop Engineering & Graph Engineering Interview Questions and Answers Architecting Production-Grade Agentic AI Systems

55 1 RAG Evaluation Metrics — Interview Questions & Answers Sanjay Kumar PhD Sanjay Kumar PhD

Mar 22

RAG Evaluation Metrics — Interview Questions & Answers ♦ 1. What are the main categories of metrics used to evaluate RAG systems?

25 See all from Sanjay Kumar PhD Help

Status

About

Careers

Press

Blog

Store

Privacy

Rules

Terms

Text to speech

